Computer Architecture

SOONGSIL UNIVERSITY 1897

Network Computing Lab.

Arm assembly Lab 1

Contents

- 실습 목표
- Assembler & Debugger 사용법 간단 소개
- Lab1 : Factorial 계산 프로그램 test
  - Factorial - non recursive
  - Factorial - recursive

- 부록 - APCS
- 부록 - ARM7: A 32 bit CPSR

실습 목표

- ARM assembly에 대한 실습을 통해 이론으로 배웠던 ARM instruction들을 실제로 다루어 본다.
- C코드 수행을 Assembly 코드 수행으로 분석한다. 변수를 위해 레지스터 및 메모리가 사용되고, 자료구조가 메모리에 할당되어 수행되는 과정을 명령어 수준에서 살펴본다.
- Debugger 사용법을 배운다
- HW 5: P40의 실습 A,B 수행 후 결과내용을 캡쳐 또는 내용 작성하여 보고서로 제출하세요.

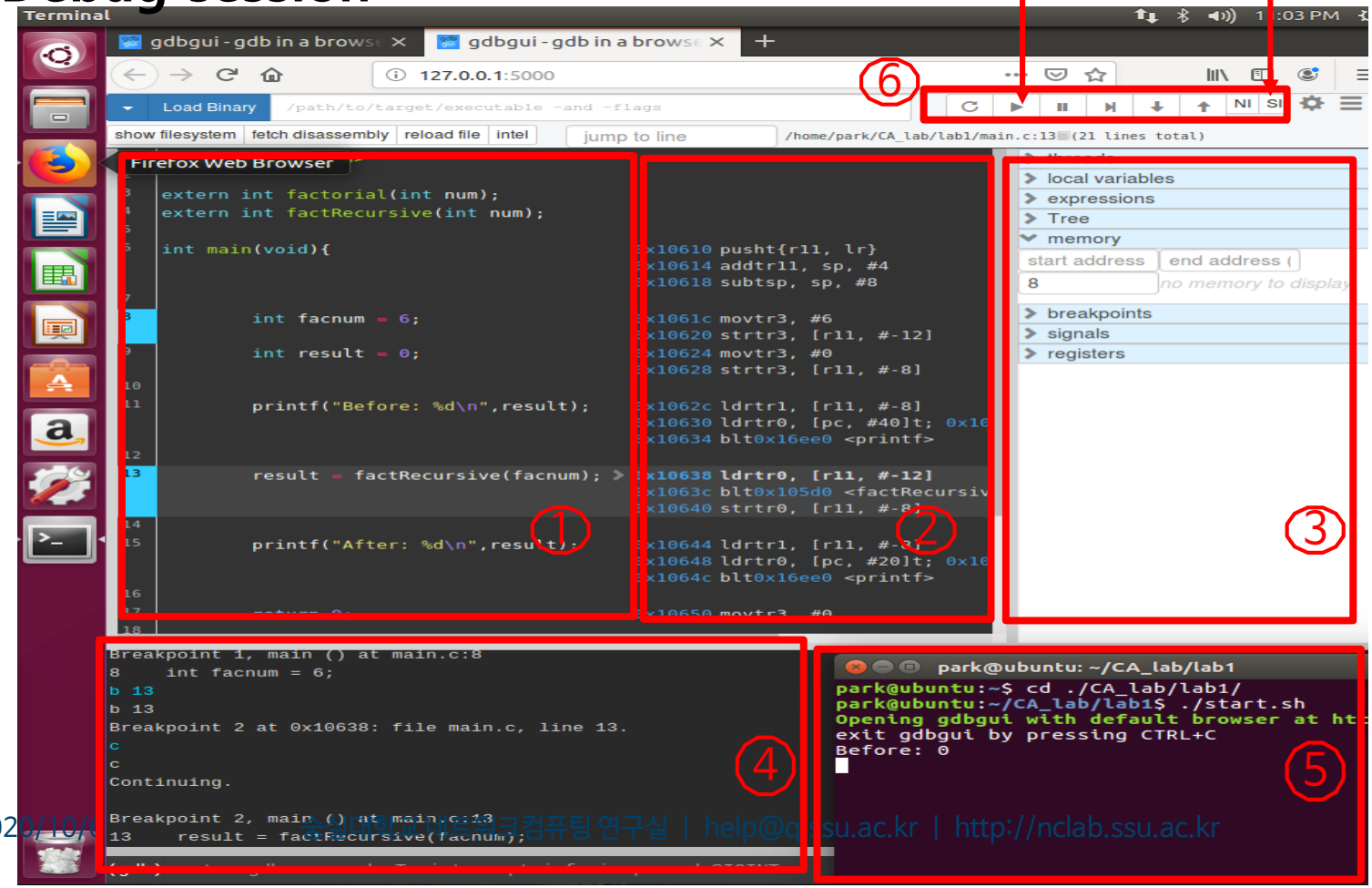
Lab 1

① Source code window : 작성한 코드를 보여준다.
② Disassembly : build해서 만든 실행 파일을 assemble code 로 표현해준다.
③ state of program : 현재 프로그램 상태에 대한 정보
④ Gdb 명령어 입력창
⑤ 결과 출력 터미널
⑥ 단축 아이콘 화면

단축아이콘 SI

단축아이콘 continue

- Debug session

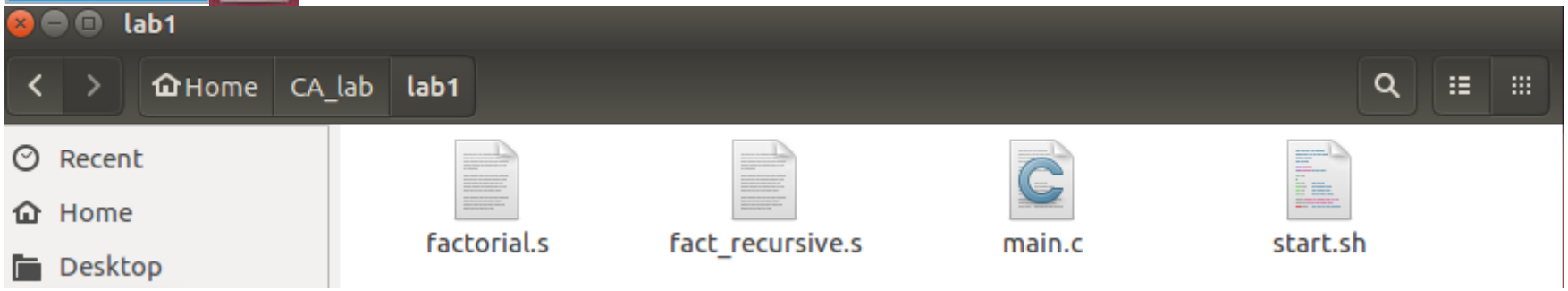


Lab 1

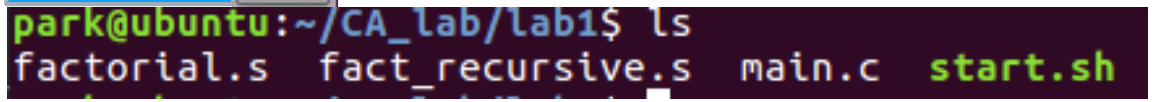
□ 새로운 source file download

- http://nclab.ssu.ac.kr/ 웹페이지에 접속
- Course->학부수업->컴퓨터구조(2020-2)->강의자료 탭에서 lab1 file을 받는다.
- 디렉토리 lab1을 생성하고 압축 file을 lab1에 푼다.

파일탐색기



터미널



Lab 1

디버깅 준비

컴파일

$ arm-linux-gnueabi-gcc -o lab1 fact_recursive.s factorial.s main.c -g -static

```
park@ubuntu:~/CA_lab/lab1$ arm-linux-gnueabi-gcc -o lab1 fac_recursive.s Factorial.s
 main.c -g -static
park@ubuntu:~/CA_lab/lab1$ ls
fac_recursive.s  Factorial.s  lab1  main.c
```

qemu-arm 실행

$ qemu-arm - g 8080 ./lab1 (lab1파일의 경로 입력)

```
park@ubuntu:~/CA_lab/lab1$ qemu-arm -g 8080 ./lab1
```

gdbgui 실행

$ gdbgui - g gdb-multiarch

```
park@ubuntu:~/CA_lab/lab1$ gdbgui -g gdb-multiarch
Opening gdbgui with default browser at http://127.0.0.1:5000
exit gdbgui by pressing CTRL+C
```

Lab 1 : Factorial – non recursive

- C code (예시코드)

```
int factorial (int n){
   int result = 1;

   for(int i=n; i > 0; i--) {
       result *= i ;
   }
   return result;
 }
```

(예) 3! = 3 * 2 * 1 = 6

Lab 1 : Factorial – non recursive

Source code

main.c

```c
#include <stdio.h>

extern int factorial(int num);
extern int factRecursive(int num);

int main(void){

          int facnum = 6;
          int result = 0;

         printf("Before: %d\n",result);

          result = factorial(facnum);

          printf("After: %d\n",result);

          return 0;

}
```

factorial.s

```
.text
.global factorial

factorial:
          push      {r4,lr}
          mov       r4, r0
Loop:
          sub       r4, r4, #1
          mul       r0, r4, R0
          cmp       r4, #1
          bgt       Loop

          pop       {r4, pc}

.end
```

(예) 3! = 3 * 2 * 1 = 6

Lab 1 : Factorial – non recursive

- 디버깅 시작
  - gdbgui 를 실행시킨 브라우저에서 gdb 명령어 입력

(gdb) file {lab1파일이 위치하는 경로}/lab1

```
(gdb) file ~/CA_lab/lab1/lab1
park@ubuntu:~/CA_lab/lab1$ ls
fac_recursive.s  Factorial.s  lab1  mai
```

or

```
(gdb) file ./lab1
```

(현재 디렉토리에 lab1이 존재 할 때)

(gdb) target remote:8080

```
target remote:8080
Remote debugging using :8080
0x0001044c in _start ()
(gdb) target remote:8080
park@ubuntu:~/CA_lab/lab1$ qemu-arm -g 8080 ./lab1
```

(gdb) b main

```
b main
Note: breakpoint 1 also set at pc 0x1061c.
Breakpoint 2 at 0x1061c: file main.c, line 6.
(gdb) b main
```

Lab 1 : Factorial – non recursive

- 디버깅
  - (gdb) c를 입력하면 main 함수에서 break

(gdb) b factorial → (gdb) c 를 입력해 factorial 함수로 진입

```c
#include <stdio.h>
extern int fact(int num);
extern int factorial(int num);
int main(void){

        int facnum = 6;
        int result = 0;

        printf("Before: %d\n",result);

        result = factorial(facnum);
        Break point

        printf("After: %d\n",result);

        return 0;
}
```

```
.text
.global factorial

factorial:
        push    {r4,lr}
        mov     r4, r0
Loop:
        sub     r4, r4, #1
        mul     r0, r4, r0
        cmp     r4, #1
        bgt     Loop

        pop     {r4, pc}
.end
```

Lab 1 : Factorial – non recursive

□ 디버깅

- **(gdb) b [파일명]:[Lable 명] 를 통해 break 설정가능**

(gdb) **b Loop** (또는 b Factorial.s:Loop)

(gdb) **c** 를 입력해 Loop로 진입 (sub 수행 전)

breakpoint

```
.text
.global factorial

factorial:
        push    {r4,lr}
        mov     r4, r0
Loop:
        sub     r4, r4, #1
        mul     r0, r4, r0
        cmp     r4, #1
        bgt     Loop

        pop     {r4, pc}
.end
```

Lab 1 : Factorial – non recursive

디버깅

registers

name	value (hex)	value (decimal)
r0	0x6	6
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0x6	6
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0
r9	0x0	0
r10	0x96f6c	618348
r11	0xf6fff024	4143968292
r12	0x0	0
sp	0xf6fff010	4143968272
lr	0x10640	67136
pc	0x105bc	67004

```
1  .text
2  .global factorial
3
4  factorial:
5          push    {r4,lr}
6          mov     r4, r0
7  Loop:
8          sub     r4, r4, #1
9          mul     r0, r4, r0
10         cmp     r4, #1
11         bgt     Loop
12
13         pop     {r4, pc}
14 .end
15
16
   (end of file)
```

other pid pid

registers

name	value (hex)	value (decimal)
r0	0x1e	30
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0x5	5
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0
r9	0x0	0
r10	0x96f6c	618348
r11	0xf6fff024	4143968292
r12	0x0	0
sp	0xf6fff010	4143968272
lr	0x10640	67136
pc	0x105c4	67012

(gdb) si 을 입력하거나 단축아이콘 SI 클릭해 명령어 단위로 실행시켜 봅시다
그럼 r0, r4, pc의 값이 바뀌는 것을 확인하면서 이런 식이구나~ 하고 이해해봅시다.

Lab 1 : Factorial – non recursive

- 디버깅

name	value (hex)	value (decimal)
r0	0x1e	30
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0x5	5
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0

```
Loop:
        sub     r4, r4, #1
        mul     r0, r4, r0
        cmp     r4, #1
        bgt     Loop
```

name	value (hex)	value (decimal)
r0	0x2d0	720
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0x1	1
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0

빨간색 박스를 위주로 값을 비교하면서
단축아이콘 SI 클릭하여 명령어 단위로 실행시켜보세요.
gt(great equal) 조건이 만족할 때까지
이 Loop를 계속 돌면서 값이 변하는 것을 확인할 수 있습니다.

하단에 CPSR 필드는 있으나 정보가 나오지 않음

- **cmp 명령어 수행시마다 (gdb) info reg cpsr을 입력해 cpsr값의 변화를 확인**
- **부록(P43) - ARM7: A 32 bit CPSR 참고**

```
cpsr            0x20000010
  536870928

(gdb)  info reg
```

N	0
Z	0
C	1
V	0

```
cpsr            0x60000010
  1610612752
```

N	0
Z	1
C	1
V	0

Lab 1 : Factorial – recursive

- C code (예시)

```
int factRecursive (int n){

   if (n < 1) return 1;
   else return n * factRecursive(n - 1);
 }
```

Lab 1 : Factorial – recursive

아래 코드는 2장 Part3 P20~23 코드와 동일

Source code

main.c

- main.c 수정 할 것

```c
#include <stdio.h>

extern int factorial(int num);
extern int factRecursive(int num);

int main(void){

        int facnum = 6;
        int result = 0;

        printf("Before: %d\n",result);

        result = factRecursive(facnum);

        printf("After: %d\n",result);

        return 0;

}
```

- factorial 에서 factRecursive로 수정

fact_recursive.s

```asm
.text
.global factRecursive

factRecursive:
        sub  sp, sp, #8
        str  lr, [sp, #4]
        str  r0, [sp, #0]
        cmp  r0, #1
        bge  loop
        mov  r0, #1
        add  sp, sp, #8
        mov  pc,lr
loop:
        sub  r0, r0, #1
        bl   factRecursive
        mov  r12, r0
        ldr  r0, [sp, #0]
        ldr  lr, [sp, #4]
        add  sp, sp, #8
        mul  r0, r12, r0
        mov  pc, lr
.end
```

Lab 1 : Factorial – recursive

- (gdb) b main

여기서, facnum local 변수는 r3 register를 사용합니다.
Break point 설정하고 (gdb)si 입력한 후 r3 값을 확인해 봅시다.
MOV r3, #6 으로 r3에 값이 대입되는 것을 확인할 수 있습니다.

show filesystem | fetch disassembly | reload file | intel | jump to line | /home/park/CA_lab/l

```
1  #include <stdio.h>
2
3  extern int factorial(int num);
4  extern int factRecursive(int num);
5
6  int main(void){
7
8  break point  int facnum = 6;
9          int result = 0;
10
11         printf("Before: %d\n",result);
12
13         result = factRecursive(facnum);
14
15         printf("After: %d\n",result);
16
17         return 0;
```

```
0x10610 pusht{r11, lr}
0x10614 addtr11, sp, #4
0x10618 subtsp, sp, #8
0x1061c movtr3, #6
0x10620 strtr3, [r11, #-12]
0x10624 movtr3, #0
0x10628 strtr3, [r11, #-8]
0x1062c ldrtr1, [r11, #-8]
0x10630 ldrtr0, [pc, #40]t;
0x10634 blt0x16ee0 <printf>
0x10638 ldrtr0, [r11, #-12]
0x1063c blt0x105d0 <factRecu
0x10640 strtr0, [r11, #-8]
0x10644 ldrtr1, [r11, #-8]
0x10648 ldrtr0, [pc, #20]t;
0x1064c blt0x16ee0 <printf>
0x10650 movtr3, #0
```

fetch disassembly를 눌러 명령어를 확인 할 수 있습니다

registers

name	value (hex)
r0	0x1
r1	0xf6fff174
r2	0xf6fff17c
r3	0x10610

registers

name	value (hex)
r0	0x1
r1	0xf6fff174
r2	0xf6fff17c
r3	0x6

Lab 1 : Factorial – recursive

- (gdb) si

```
add r11, sp, #4
sub sp , sp, #8
.. .
str r3, [r11, #-12]
```

r11 값의 의미 파악

```
r11 = sp + 4
r11 = sp + 12

r11 – 12 = sp
```

r11	0xf6fff024	4143968292
r12	0x0	0
sp	0xf6fff018	4143968280

show filesystem | fetch disassembly | reload file | intel | jump to line

```
1  #include <stdio.h>
2
3  extern int factorial(int num);
4  extern int factRecursive(int num);
5
6  int main(void){                          0x10610 pusht{r11, lr}
                                            0x10614 addtr11, sp, #4
                                            0x10618 subtsp, sp, #8
7
8          int facnum = 6;                  0x1061c movtr3, #6
                                            0x1062c strtr3, [r11, #-12]
9          int result = 0;                  0x10624 movtr3, #0
                                            0x10628 strtr3, [r11, #-8]
10
11         printf("Before: %d\n",result);   0x1062c ldrtr1, [r11, #-8]
                                            0x10630 ldrtr0, [pc, #40]t;
                                            0x10634 blt0x16ee0 <printf>
12
13         result = factRecursive(facnum);  0x10638 ldrtr0, [r11, #-12]
                                            0x1063c blt0x105d0 <factRecu
                                            0x10640 strtr0, [r11, #-8]
14
15         printf("After: %d\n",result);    0x10644 ldrtr1, [r11, #-8]
                                            0x10648 ldrtr0, [pc, #20]t;
                                            0x1064c blt0x16ee0 <printf>
16
17         return 0;                        0x10658 movtr3, #0
```

sp 0xf6fff018 — 눌러서 메모리 값 조회

memory: 0xf6fff018 – 0xf6fff043, 4

address	hex
more	
0xf6fff018	00 00 00 00
0xf6fff01c	00 00 00 00
0xf6fff020	00 00 00 00
0xf6fff024	fc 08 01 00

memory: 0xf6fff018 – 0xf6fff043, 4

address	hex
more	
0xf6fff018	06 00 00 00
0xf6fff01c	00 00 00 00
0xf6fff020	00 00 00 00
0xf6fff024	fc 08 01 00

Stack에 r3값이 저장되는 것을 확인

Lab 1 : Factorial – recursive

- (gdb) b 13 , (gdb) c

factRecursive() 함수를 호출하고 argument로 facnum을 전달하고 있습니다. 어떻게 assembly code로 세 줄이 표현되었는지 생각해봅시다. 그냥 factRecursive() 함수로 branch하면 되지 왜 r0 register에 어떤 값을 넣고 이후 r0 reg에서 뭘 또 다시 옮길까요?
본론부터 말하면, factRecursive() 함수의 argument 값을 넘겨주는 동작과 리턴 값을 다른 local 변수에 받는 동작 때문입니다.
이는 APCS(ARM Procedure Call Standard)에 관련한 내용입니다.

* P41 APCS 관련 부록을 참고하세요.

```
1  #include <stdio.h>
2
3  extern int factorial(int num);
4  extern int factRecursive(int num);
5
6  int main(void){                          0x10610 pusht{r11, lr}
                                            0x10614 addtr11, sp, #4
                                            0x10618 subtsp, sp, #8
7
8          int facnum = 6;                  0x1061c movtr3, #6
                                            0x10620 strtr3, [r11, #-12]
9          int result = 0;                  0x10624 movtr3, #0
                                            0x10628 strtr3, [r11, #-8]
10
11         printf("Before: %d\n",result);   0x1062c ldrtr1, [r11, #-8]
                                            0x10630 ldrtr0, [pc, #40]t; 0x10660
                                            0x10634 blt0x16ee0 <printf>
12
13         result = factRecursive(facnum);  0x10638 ldrtr0, [r11, #-12]
                                            0x1063c blt0x105d0 <factRecursive>
                                            0x10640 strtr0, [r11, #-8]
14
15         printf("After: %d\n",result);    0x10644 ldrtr1, [r11, #-8]
                                            0x10648 ldrtr0, [pc, #20]t; 0x10664
                                            0x1064c blt0x16ee0 <printf>
16
17         return 0;                        0x10650 movtr3, #0
```

break point

ldr r0, [r11, #-12]

memory

0xf6fff018 0xf6fff043

4

address	hex
more	
0xf6fff018	06 00 00 00
0xf6fff01c	00 00 00 00

[r11 – 12], 즉 sp가 가르키고 있는 곳의 값을 읽어와 r0 에 넣습니다.

Lab 1 : Factorial – recursive

- **Debugging**
  - (gdb)si 를 입력해서 확인

```
result = factRecursive(facnum);  ① 0x10638 ldrtr0, [r11, #-12]
                                 ② 0x1063c blt0x105d0 <factRecursive>
                                   0x10640 strtr0, [r11, #-8]
```

factRecursive함수의 argument가 되는 Facnum의 값

factRecursive함수의 주소를 pc에 넣어 branch

registers

name	value (hex)	value (decimal)
r0	0xa	10
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0xf6fff038	4143968312
⋮		
sp	0xf6fff018	4143968280
lr	0x10638	67128
pc	0x10638	67128

registers

name	value (hex)	value (decimal)
① r0	0x6	6
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0xf6fff038	4143968312
⋮		
sp	0xf6fff018	4143968280
lr	0x10638	67128
pc	0x1063c	67132

registers

name	value (hex)	value (decimal)
r0	0x6	6
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0xf6fff038	4143968312
⋮		
sp	0xf6fff018	4143968280
lr	0x10640	67136
② pc	0x105d0	67024

Lab 1 : Factorial – recursive

Debugging

- b factRecursive 후 si로 진행

break point

```
.text
.global factRecursive

factRecursive:
        sub     sp, sp, #8
        str     lr, [sp, #4]
        str     r0, [sp, #0]
        cmp     r0, #1
        bge     loop
        mov     r0, #1
        add     sp, sp, #8
        mov     pc,lr
loop:
        sub     r0, r0, #1
        bl      factRecursive
        mov     r12, r0
        ldr     r0, [sp, #0]
        ldr     lr, [sp, #4]
        add     sp, sp, #8
        mul     r0, r12, r0
        mov     pc, lr
.end
```

sp 0xf6fff010

파란색 값을 눌러 memory변화를 확인합니다.

```
str     lr, [sp, #4]
str     r0, [sp, #0]
```

memory

0xf6fff010 | 0xf6fff02f

8

address	hex
more	
0xf6fff010	7c f1 ff f6
	00 00 00 00
0xf6fff018	06 00 00 00
	00 00 00 00
0xf6fff020	00 00 00 00
	fc 08 01 00
0xf6fff028	00 00 00 00
	01 00 00 00

memory

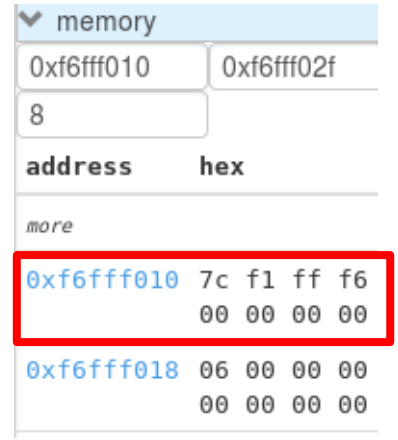
0xf6fff010 | 0xf6fff02f

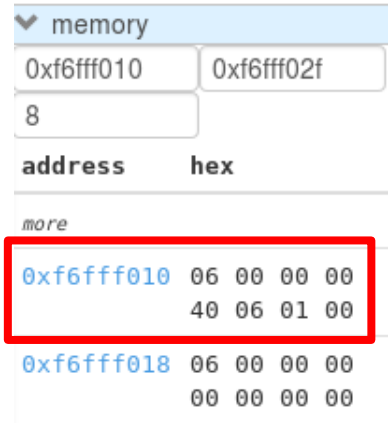
8

address	hex
more	
0xf6fff010	06 00 00 00
	40 06 01 00
0xf6fff018	06 00 00 00
	00 00 00 00
0xf6fff020	00 00 00 00
	fc 08 01 00
0xf6fff028	00 00 00 00
	01 00 00 00

Lab 1 : Factorial – recursive

- **Debugging**

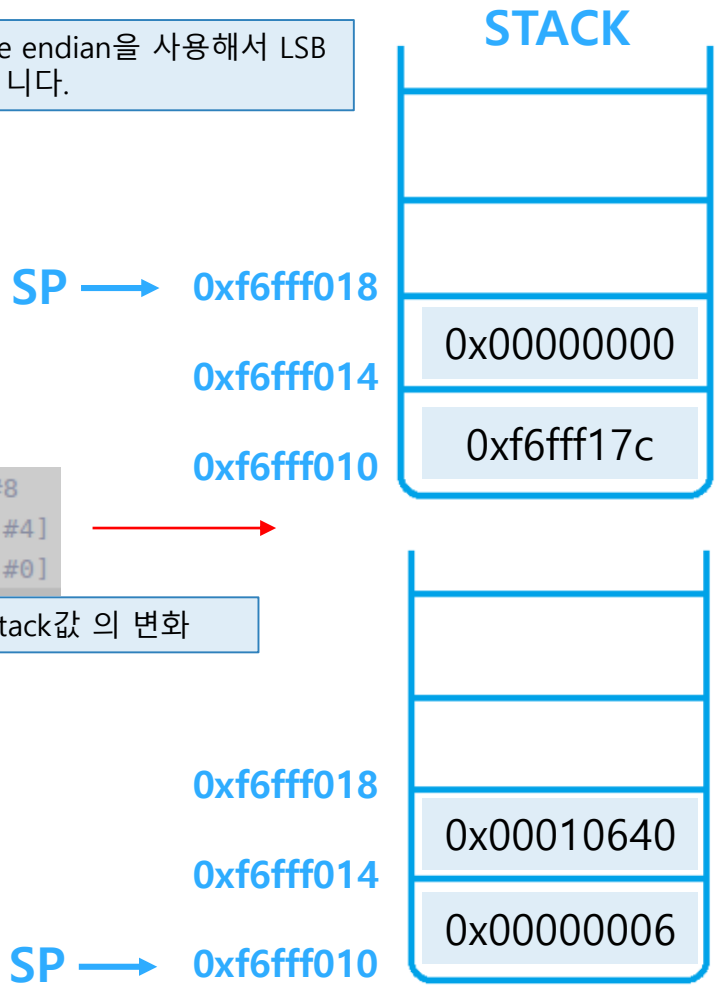




현재 사용 pc는 Little endian을 사용해서 LSB 가 먼저 스택에 쌓입니다.

```
sub     sp, sp, #8
str     lr, [sp, #4]
str     r0, [sp, #0]
```

위 명령어 수행 시 stack값 의 변화



Lab 1 : Factorial – recursive

- (gdb) b loop로 break 설정

```
1   .text
2   .global factRecursive
3
4   factRecursive:
5           sub     sp, sp, #8
6           str     lr, [sp, #4]
7           str     r0, [sp, #0]
8           cmp     r0, #1
9           bge     loop
10          mov     r0, #1
11          add     sp, sp, #8
12          mov     pc,lr
13  loop:
14          sub     r0, r0, #1
15          bl      factRecursive
16          mov     r12, r0
17          ldr     r0, [sp, #0]
18          ldr     lr, [sp, #4]
19          add     sp, sp, #8
20          mul     r0, r12, r0
21          mov     pc, lr
22  .end
```

break point

sub명령어에 break point로 설정하고 (gdb) c 를 한번씩 입력하거나 단축아이콘 Continue를 클릭하면서, r0의 값의 변화를 확인해 보세요.

Lab 1 : Factorial – recursive

Debugging

recursive call

```
.text
.global factRecursive

factRecursive:
        sub     sp, sp, #8
        str     lr, [sp, #4]
        str     r0, [sp, #0]
        cmp     r0, #1
        bge     loop
        mov     r0, #1
        add     sp, sp, #8
        mov     pc,lr
loop:
        sub     r0, r0, #1
        bl      factRecursive
        mov     r12, r0
        ldr     r0, [sp, #0]
        ldr     lr, [sp, #4]
        add     sp, sp, #8
        mul     r0, r12, r0
        mov     pc, lr
.end
```

R0 >= 1 ?

yes

no

(gdb) c 를 입력하면 이런 sequence로 동작할 겁니다. 계속 r0값과 1을 비교하죠.
factorial 연산하기 위해 초기 값에서 1까지 계속 비교하는 동작입니다.
아까 보여드린 C code에서 if 조건문과 다시 fact함수를 recursive call하는 동작을 하고 있는 겁니다.

```
int factRecursive(int n){

  if (n < 1) return 1;
  else return n * factRecursive(n - 1);
}
```

Lab 1 : Factorial – recursive

- (gdb) c 를 입력해 진행하면서 변화를 확인

```
.text
.global factRecursive

factRecursive:
        sub     sp, sp, #8
        str     lr, [sp, #4]
        str     r0, [sp, #0]
        cmp     r0, #1
        bge     loop
        mov     r0, #1
        add     sp, sp, #8
        mov     pc,lr
loop:
        sub     r0, r0, #1
        bl      factRecursive
        mov     r12, r0
        ldr     r0, [sp, #0]
        ldr     lr, [sp, #4]
        add     sp, sp, #8
        mul     r0, r12, r0
        mov     pc, lr
.end
```

0xf6ffefe4	00 00 00 00
0xf6ffefe8	00 00 00 00
0xf6ffefec	6c 6f 09 00
0xf6ffeff0	24 f0 ff f6
0xf6ffeff4	08 6f 01 00
0xf6ffeff8	58 0e 01 00
0xf6ffeffc	0c f0 ff f6
0xf6fff000	01 00 00 00
0xf6fff004	38 06 01 00
0xf6fff008	dc 12 07 00
0xf6fff00c	00 00 00 00
0xf6fff010	06 00 00 00
0xf6fff014	40 06 01 00

r0	0x6	6
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0xf6fff038	4143968312
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0
r9	0x0	0
r10	0x96f6c	618348
r11	0xf6fff024	4143968292
r12	0x0	0
sp	0xf6fff010	4143968272
lr	0x10640	67136
pc	0x105dc	67036

Lab 1 : Factorial – recursive

Debugging

```
1  .text
2  .global factRecursive
3
4  factRecursive:
5          sub     sp, sp, #8
6          str     lr, [sp, #4]
7          str     r0, [sp, #0]
8          cmp     r0, #1
9          bge     loop
10         mov     r0, #1
11         add     sp, sp, #8
12         mov     pc,lr
13 loop:
14         sub     r0, r0, #1
15         bl      factRecursive
16         mov     r12, r0
17         ldr     r0, [sp, #0]
18         ldr     lr, [sp, #4]
19         add     sp, sp, #8
20         mul     r0, r12, r0
21         mov     pc, lr
22 .end
```

recursive call

0xf6ffefe4	00 00 00 00
0xf6ffefe8	00 00 00 00
0xf6ffefec	6c 6f 09 00
0xf6ffeff0	24 f0 ff f6
0xf6ffeff4	08 6f 01 00
0xf6ffeff8	58 0e 01 00
0xf6ffeffc	0c f0 ff f6
0xf6fff000	01 00 00 00
0xf6fff004	38 06 01 00
0xf6fff008	05 00 00 00
0xf6fff00c	f8 05 01 00
0xf6fff010	06 00 00 00
0xf6fff014	40 06 01 00

r0	0x5	5
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0xf6fff038	4143968312
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0
r9	0x0	0
r10	0x96f6c	618348
r11	0xf6fff024	4143968292
r12	0x0	0
sp	0xf6fff010	4143968272
lr	0x105f8	67064
pc	0x105d0	67024

Lab 1 : Factorial – recursive

Debugging

```
1   .text
2   .global factRecursive
3
4   factRecursive:
5           sub     sp, sp, #8
6           str     lr, [sp, #4]
7           str     r0, [sp, #0]
8           cmp     r0, #1
9           bge     loop
10          mov     r0, #1
11          add     sp, sp, #8
12          mov     pc,lr
13  loop:
14          sub     r0, r0, #1
15          bl      factRecursive
16          mov     r12, r0
17          ldr     r0, [sp, #0]
18          ldr     lr, [sp, #4]
19          add     sp, sp, #8
20          mul     r0, r12, r0
21          mov     pc, lr
22  .end
```

recursive call

0xf6ffefe0	00 00 00 00
0xf6ffefe4	00 00 00 00
0xf6ffefe8	00 00 00 00
0xf6ffefec	6c 6f 09 00
0xf6ffeff0	24 f0 ff f6
0xf6ffeff4	08 6f 01 00
0xf6ffeff8	58 0e 01 00
0xf6ffeffc	0c f0 ff f6
0xf6fff000	04 00 00 00
0xf6fff004	f8 05 01 00
0xf6fff008	05 00 00 00
0xf6fff00c	f8 05 01 00
0xf6fff010	06 00 00 00
0xf6fff014	40 06 01 00

r0	0x4	4
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0xf6fff038	4143968312
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0
r9	0x0	0
r10	0x96f6c	618348
r11	0xf6fff024	4143968292
r12	0x0	0
sp	0xf6fff008	4143968264
lr	0x105f8	67064
pc	0x105d0	67024

Lab 1 : Factorial – recursive

Debugging

recursive call

```
.text
.global factRecursive

factRecursive:
        sub     sp, sp, #8
        str     lr, [sp, #4]
        str     r0, [sp, #0]
        cmp     r0, #1
        bge     loop
        mov     r0, #1
        add     sp, sp, #8
        mov     pc,lr
loop:
        sub     r0, r0, #1
        bl      factRecursive
        mov     r12, r0
        ldr     r0, [sp, #0]
        ldr     lr, [sp, #4]
        add     sp, sp, #8
        mul     r0, r12, r0
        mov     pc, lr
.end
```

0xf6ffefe0	00 00 00 00
0xf6ffefe4	00 00 00 00
0xf6ffefe8	00 00 00 00
0xf6ffefec	6c 6f 09 00
0xf6ffeff0	24 f0 ff f6
0xf6ffeff4	08 6f 01 00
0xf6ffeff8	03 00 00 00
0xf6ffeffc	f8 05 01 00
0xf6fff000	04 00 00 00
0xf6fff004	f8 05 01 00
0xf6fff008	05 00 00 00
0xf6fff00c	f8 05 01 00
0xf6fff010	06 00 00 00
0xf6fff014	40 06 01 00

r0	0x3	3
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0xf6fff038	4143968312
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0
r9	0x0	0
r10	0x96f6c	618348
r11	0xf6fff024	4143968292
r12	0x0	0
sp	0xf6fff000	4143968256
lr	0x105f8	67064
pc	0x105d0	67024

Lab 1 : Factorial – recursive

Debugging

```
1   .text
2   .global factRecursive
3
4   factRecursive:
5           sub     sp, sp, #8
6           str     lr, [sp, #4]
7           str     r0, [sp, #0]
8           cmp     r0, #1
9           bge     loop
10          mov     r0, #1
11          add     sp, sp, #8
12          mov     pc,lr
13  loop:
14          sub     r0, r0, #1
15          bl      factRecursive
16          mov     r12, r0
17          ldr     r0, [sp, #0]
18          ldr     lr, [sp, #4]
19          add     sp, sp, #8
20          mul     r0, r12, r0
21          mov     pc, lr
22  .end
```

recursive call

0xf6ffefe0	00 00 00 00
0xf6ffefe4	00 00 00 00
0xf6ffefe8	00 00 00 00
0xf6ffefec	6c 6f 09 00
0xf6ffeff0	02 00 00 00
0xf6ffeff4	f8 05 01 00
0xf6ffeff8	03 00 00 00
0xf6ffeffc	f8 05 01 00
0xf6fff000	04 00 00 00
0xf6fff004	f8 05 01 00
0xf6fff008	05 00 00 00
0xf6fff00c	f8 05 01 00
0xf6fff010	06 00 00 00
0xf6fff014	40 06 01 00

r0	0x2	2
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0xf6fff038	4143968312
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0
r9	0x0	0
r10	0x96f6c	618348
r11	0xf6fff024	4143968292
r12	0x0	0
sp	0xf6ffeff8	4143968248
lr	0x105f8	67064
pc	0x105d0	67024

Lab 1 : Factorial – recursive

Debugging

```
1   .text
2   .global factRecursive
3
4   factRecursive:
5           sub     sp, sp, #8
6           str     lr, [sp, #4]
7           str     r0, [sp, #0]
8           cmp     r0, #1
9           bge     loop
10          mov     r0, #1
11          add     sp, sp, #8
12          mov     pc,lr
13  loop:
14          sub     r0, r0, #1
15          bl      factRecursive
16          mov     r12, r0
17          ldr     r0, [sp, #0]
18          ldr     lr, [sp, #4]
19          add     sp, sp, #8
20          mul     r0, r12, r0
21          mov     pc, lr
22  .end
```

recursive call

0xf6ffefe0	00 00 00 00
0xf6ffefe4	00 00 00 00
0xf6ffefe8	01 00 00 00
0xf6ffefec	f8 05 01 00
0xf6ffeff0	02 00 00 00
0xf6ffeff4	f8 05 01 00
0xf6ffeff8	03 00 00 00
0xf6ffeffc	f8 05 01 00
0xf6fff000	04 00 00 00
0xf6fff004	f8 05 01 00
0xf6fff008	05 00 00 00
0xf6fff00c	f8 05 01 00
0xf6fff010	06 00 00 00
0xf6fff014	40 06 01 00

r0	0x1	1
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0xf6fff038	4143968312
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0
r9	0x0	0
r10	0x96f6c	618348
r11	0xf6fff024	4143968292
r12	0x0	0
sp	0xf6ffeff0	4143968240
lr	0x105f8	67064
pc	0x105d0	67024

Lab 1 : Factorial – recursive

- (gdb) **b 21**

```
1   .text
2   .global factRecursive
3
4   factRecursive:
5           sub     sp, sp, #8
6           str     lr, [sp, #4]
7           str     r0, [sp, #0]
8           cmp     r0, #1
9           bge     loop
10          mov     r0, #1
11          add     sp, sp, #8
12          mov     pc,lr
13  loop:
14          sub     r0, r0, #1
15          bl      factRecursive
16          mov     r12, r0
17          ldr     r0, [sp, #0]
18          ldr     lr, [sp, #4]
19          add     sp, sp, #8
20          mul     r0, r12, r0
21          mov     pc, lr
22  .end
```

breakpoint

이제 추가로 빨간 점으로 찍혀 있는 곳을 break point로 설정하고 설정한 지점에 도착할 때 까지 진행해보세요.

Lab 1 : Factorial – recursive

Debugging

```
.text
.global factRecursive

factRecursive:
        sub     sp, sp, #8
        str     lr, [sp, #4]
        str     r0, [sp, #0]
        cmp     r0, #1
        bge     loop
        mov     r0, #1
        add     sp, sp, #8
        mov     pc,lr
loop:
        sub     r0, r0, #1
        bl      factRecursive
        mov     r12, r0
        ldr     r0, [sp, #0]
        ldr     lr, [sp, #4]
        add     sp, sp, #8
        mul     r0, r12, r0
        mov     pc, lr
.end
```

0xf6ffefe0	00 00 00 00
0xf6ffefe4	f8 05 01 00
0xf6ffefe8	01 00 00 00
0xf6ffefec	f8 05 01 00
0xf6ffeff0	02 00 00 00
0xf6ffeff4	f8 05 01 00
0xf6ffeff8	03 00 00 00
0xf6ffeffc	f8 05 01 00
0xf6fff000	04 00 00 00
0xf6fff004	f8 05 01 00
0xf6fff008	05 00 00 00
0xf6fff00c	f8 05 01 00
0xf6fff010	06 00 00 00
0xf6fff014	40 06 01 00

r0	0x0	0
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0xf6fff038	4143968312
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0
r9	0x0	0
r10	0x96f6c	618348
r11	0xf6fff024	4143968292
r12	0x0	0
sp	0xf6ffefe8	4143968232
lr	0x105f8	67064
pc	0x105d0	67024

Lab 1 : Factorial – recursive

- (gdb) b 21

```
1   .text
2   .global factRecursive
3
4   factRecursive:
5           sub     sp, sp, #8
6           str     lr, [sp, #4]
7           str     r0, [sp, #0]
8           cmp     r0, #1
9           bge     loop
10          mov     r0, #1
11          add     sp, sp, #8
12          mov     pc,lr
13  loop:
14          sub     r0, r0, #1
15          bl      factRecursive
16          mov     r12, r0
17          ldr     r0, [sp, #0]
18          ldr     lr, [sp, #4]
19          add     sp, sp, #8
20          mul     r0, r12, r0
21          mov     pc, lr
22  .end
```

breakpoint

이제 (gdb) si를 누르면서 r0, r12, lr register 의 값을 확인해보세요.

Lab 1 : Factorial – recursive

Debugging

```
1   .text
2   .global factRecursive
3
4   factRecursive:
5           sub     sp, sp, #8
6           str     lr, [sp, #4]
7           str     r0, [sp, #0]
8           cmp     r0, #1
9           bge     loop
10          mov     r0, #1
11          add     sp, sp, #8
12          mov     pc,lr
13  loop:
14          sub     r0, r0, #1
15  Return  bl      factRecursive
16          mov     r12, r0
17          ldr     r0, [sp, #0]
18          ldr     lr, [sp, #4]
19          add     sp, sp, #8
20          mul     r0, r12, r0
21          mov     pc, lr
22  .end
```

0xf6ffefe0	00 00 00 00
0xf6ffefe4	f8 05 01 00
0xf6ffefe8	01 00 00 00
0xf6ffefec	f8 05 01 00
0xf6ffeff0	02 00 00 00
0xf6ffeff4	f8 05 01 00
0xf6ffeff8	03 00 00 00
0xf6ffeffc	f8 05 01 00
0xf6fff000	04 00 00 00
0xf6fff004	f8 05 01 00
0xf6fff008	05 00 00 00
0xf6fff00c	f8 05 01 00
0xf6fff010	06 00 00 00
0xf6fff014	40 06 01 00

r0	0x1	1
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0xf6fff038	4143968312
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0
r9	0x0	0
r10	0x96f6c	618348
r11	0xf6fff024	4143968292
r12	0x1	1
sp	0xf6ffeff0	4143968240
lr	0x105f8	67064
pc	0x1060c	67084

Lab 1 : Factorial – recursive

Debugging

```
1   .text
2   .global factRecursive
3
4   factRecursive:
5           sub     sp, sp, #8
6           str     lr, [sp, #4]
7           str     r0, [sp, #0]
8           cmp     r0, #1
9           bge     loop
10          mov     r0, #1
11          add     sp, sp, #8
12          mov     pc,lr
13  loop:
14          sub     r0, r0, #1
15  Return  bl      factRecursive
16          mov     r12, r0
17          ldr     r0, [sp, #0]
18          ldr     lr, [sp, #4]
19          add     sp, sp, #8
20          mul     r0, r12, r0
21          mov     pc, lr
22  .end
```

0xf6ffefe0	00 00 00 00
0xf6ffefe4	f8 05 01 00
0xf6ffefe8	01 00 00 00
0xf6ffefec	f8 05 01 00
0xf6ffeff0	02 00 00 00
0xf6ffeff4	f8 05 01 00
0xf6ffeff8	03 00 00 00
0xf6ffeffc	f8 05 01 00
0xf6fff000	04 00 00 00
0xf6fff004	f8 05 01 00
0xf6fff008	05 00 00 00
0xf6fff00c	f8 05 01 00
0xf6fff010	06 00 00 00
0xf6fff014	40 06 01 00

r0	0x2	2
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0xf6fff038	4143968312
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0
r9	0x0	0
r10	0x96f6c	618348
r11	0xf6fff024	4143968292
r12	0x1	1
sp	0xf6ffeff8	4143968248
lr	0x105f8	67064
pc	0x1060c	67084

Lab 1 : Factorial – recursive

Debugging

```
1   .text
2   .global factRecursive
3
4   factRecursive:
5           sub     sp, sp, #8
6           str     lr, [sp, #4]
7           str     r0, [sp, #0]
8           cmp     r0, #1
9           bge     loop
10          mov     r0, #1
11          add     sp, sp, #8
12          mov     pc,lr
13  loop:
14          sub     r0, r0, #1
15  Return  bl      factRecursive
16          mov     r12, r0
17          ldr     r0, [sp, #0]
18          ldr     lr, [sp, #4]
19          add     sp, sp, #8
20          mul     r0, r12, r0
21          mov     pc, lr
22  .end
```

0xf6ffefe0	00 00 00 00	
0xf6ffefe4	f8 05 01 00	
0xf6ffefe8	01 00 00 00	
0xf6ffefec	f8 05 01 00	
0xf6ffeff0	02 00 00 00	
0xf6ffeff4	f8 05 01 00	
0xf6ffeff8	03 00 00 00	
0xf6ffeffc	f8 05 01 00	
0xf6fff000	04 00 00 00	
0xf6fff004	f8 05 01 00	
0xf6fff008	05 00 00 00	
0xf6fff00c	f8 05 01 00	
0xf6fff010	06 00 00 00	
0xf6fff014	40 06 01 00	

r0	0x6	6
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0xf6fff038	4143968312
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0
r9	0x0	0
r10	0x96f6c	618348
r11	0xf6fff024	4143968292
r12	0x2	2
sp	0xf6fff000	4143968256
lr	0x105f8	67064
pc	0x1060c	67084

Lab 1 : Factorial – recursive

Debugging

```
1   .text
2   .global factRecursive
3
4   factRecursive:
5           sub     sp, sp, #8
6           str     lr, [sp, #4]
7           str     r0, [sp, #0]
8           cmp     r0, #1
9           bge     loop
10          mov     r0, #1
11          add     sp, sp, #8
12          mov     pc,lr
13  loop:
14          sub     r0, r0, #1
15  Return  bl      factRecursive
16          mov     r12, r0
17          ldr     r0, [sp, #0]
18          ldr     lr, [sp, #4]
19          add     sp, sp, #8
20          mul     r0, r12, r0
21          mov     pc, lr
22  .end
```

0xf6ffefe0	00 00 00 00
0xf6ffefe4	f8 05 01 00
0xf6ffefe8	01 00 00 00
0xf6ffefec	f8 05 01 00
0xf6ffeff0	02 00 00 00
0xf6ffeff4	f8 05 01 00
0xf6ffeff8	03 00 00 00
0xf6ffeffc	f8 05 01 00
0xf6fff000	04 00 00 00
0xf6fff004	f8 05 01 00
0xf6fff008	05 00 00 00
0xf6fff00c	f8 05 01 00
0xf6fff010	06 00 00 00
0xf6fff014	40 06 01 00

r0	0x18	24
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0xf6fff038	4143968312
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0
r9	0x0	0
r10	0x96f6c	618348
r11	0xf6fff024	4143968292
r12	0x6	6
sp	0xf6fff008	4143968264
lr	0x105f8	67064
pc	0x1060c	67084

Lab 1 : Factorial – recursive

Debugging

```
1   .text
2   .global factRecursive
3
4   factRecursive:
5           sub     sp, sp, #8
6           str     lr, [sp, #4]
7           str     r0, [sp, #0]
8           cmp     r0, #1
9           bge     loop
10          mov     r0, #1
11          add     sp, sp, #8
12          mov     pc,lr
13  loop:
14          sub     r0, r0, #1
15  Return  bl      factRecursive
16          mov     r12, r0
17          ldr     r0, [sp, #0]
18          ldr     lr, [sp, #4]
19          add     sp, sp, #8
20          mul     r0, r12, r0
21          mov     pc, lr
22  .end
```

0xf6ffefe0	00 00 00 00
0xf6ffefe4	f8 05 01 00
0xf6ffefe8	01 00 00 00
0xf6ffefec	f8 05 01 00
0xf6ffeff0	02 00 00 00
0xf6ffeff4	f8 05 01 00
0xf6ffeff8	03 00 00 00
0xf6ffeffc	f8 05 01 00
0xf6fff000	04 00 00 00
0xf6fff004	f8 05 01 00
0xf6fff008	05 00 00 00
0xf6fff00c	f8 05 01 00
0xf6fff010	06 00 00 00
0xf6fff014	40 06 01 00

r0	0x78	120
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0xf6fff038	4143968312
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0
r9	0x0	0
r10	0x96f6c	618348
r11	0xf6fff024	4143968292
r12	0x18	24
sp	0xf6fff010	4143968272
lr	0x105f8	67064
pc	0x1060c	67084

Lab 1 : Factorial – recursive

Debugging

```
1   .text
2   .global factRecursive
3
4   factRecursive:
5           sub     sp, sp, #8
6           str     lr, [sp, #4]
7           str     r0, [sp, #0]
8           cmp     r0, #1
9           bge     loop
10          mov     r0, #1
11          add     sp, sp, #8
12          mov     pc,lr
13  loop:
14          sub     r0, r0, #1
15  Return  bl      factRecursive
16          mov     r12, r0
17          ldr     r0, [sp, #0]
18          ldr     lr, [sp, #4]
19          add     sp, sp, #8
20          mul     r0, r12, r0
21          mov     pc, lr
22  .end
```

0xf6ffefe0	00 00 00 00
0xf6ffefe4	f8 05 01 00
0xf6ffefe8	01 00 00 00
0xf6ffefec	f8 05 01 00
0xf6ffeff0	02 00 00 00
0xf6ffeff4	f8 05 01 00
0xf6ffeff8	03 00 00 00
0xf6ffeffc	f8 05 01 00
0xf6fff000	04 00 00 00
0xf6fff004	f8 05 01 00
0xf6fff008	05 00 00 00
0xf6fff00c	f8 05 01 00
0xf6fff010	06 00 00 00
0xf6fff014	40 06 01 00

r0	0x2d0	720
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0xf6fff038	4143968312
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0
r9	0x0	0
r10	0x96f6c	618348
r11	0xf6fff024	4143968292
r12	0x78	120
sp	0xf6fff018	4143968280
lr	0x10640	67136
pc	0x1060c	67084

Lab 1 : Factorial – recursive

Debugging

```
1   .text
2   .global factRecursive
3
4   factRecursive:
5           sub     sp, sp, #8
6           str     lr, [sp, #4]
7           str     r0, [sp, #0]
8           cmp     r0, #1
9           bge     loop
10          mov     r0, #1
11          add     sp, sp, #8
12          mov     pc,lr
13  loop:
14          sub     r0, r0, #1
15          bl      factRecursive
16          mov     r12, r0
17          ldr     r0, [sp, #0]
18          ldr     lr, [sp, #4]
19          add     sp, sp, #8
20          mul     r0, r12, r0
21          mov     pc, lr
22  .end
23
24
```

0x10640 :

```
int facnum = 6;                 0x1061c movtr3, #6
                                0x10620 strtr3, [r11, #-12]
int result = 0;                 0x10624 movtr3, #0
                                0x10628 strtr3, [r11, #-8]

printf("Before: %d\n",result);  0x1062c ldrtr1, [r11, #-8]
                                0x10630 ldrtr0, [pc, #40]t; 0
                                0x10634 blt0x16ee0 <printf>

result = factRecursive(facnum); 0x10638 ldrtr0, [r11, #-12]
                                0x1063c blt0x105d0 <factRecur
                              > 0x10640 strtr0, [r11, #-8]

printf("After: %d\n",result);   0x10644 ldrtr1, [r11, #-8]
                                0x10648 ldrtr0, [pc, #20]t; 0
                                0x1064c blt0x16ee0 <printf>
```

결과값

r0	0x2d0	720
r1	0x0	0
r2	0x983e8	623592
r3	0x0	0
r4	0xf6fff038	4143968312
r5	0x10dbc	69052
r6	0x0	0
r7	0x0	0
r8	0x0	0
r9	0x0	0
r10	0x96f6c	618348
r11	0xf6fff024	4143968292
r12	0x78	120
sp	0xf6fff018	4143968280
lr	0x10640	67136
pc	0x1060c	67084

APCS에 의해 R0에 결과 값을 가지고 fact함수로 back합니다. 이때 branch당시 저장(BL명령어로 인해)했던 lr값을 pc값에 넣음으로써 호출 당시의 상황으로 돌아가는 것을 확인해 봅시다. (gdb) si 를 입력해봅시다.

Lab 1 : Factorial – recursive

- **실습 A : p33(factorial-recursive)에서 하나의 loop를 아래 설명처럼 line 단위로 실행시켜 레지스터, 메모리 값을 캡쳐한 후 값이 변하는 부분을 체크해서 첨부하세요.**
  - facnum 초기값은 6이라고 하자. r0을 통해 어셈블리코드에 전달된다. P32 코드 5번 줄에 breakpoint를 설정한다. r0 = 2 가 될 때까지 go한다. r0=2일 때 5번줄부터 single instruction으로 한 line씩 수행하여 loop 코드 내 21번 line이 수행될 때까지 계속 추적한다
  - 또한, 왜 그렇게 변하는지 assembly 명령어를 해석하여 설명하시오.
- **실습 B : 실습 A 에서 8번, 9번 코드 수행 후 10번 코드 또는 14번 코드를 수행하게 되는데 어떤 상황에서 10번 코드를 수행하게 되는지 이유를 CPSR을 이용해 설명하시오.**

부록 - APCS

- **ARM Procedure Call Standard**

레지스터	APCS	역 할	비 고
r0	a1	argument 1 / integer result / scratch register	Caller save
r1	a2	argument 2 / scratch register	Caller save
r2	a3	argument 3 / scratch register	Caller save
r3	a4	argument 4 / scratch register	Caller save
r4	v1	register variable 1	Callee save
r5	v2	register variable 2	Callee save
r6	v3	register variable 3	Callee save
r7	v4	register variable 4	Callee save
r8	v5	register variable 5	Callee save
r9	sb/v6	static base / register variable 6	Callee save
r10	sl/v7	stack limit / register variable 7	Callee save
r11	fp	frame pointer	Callee save
r12	ip	scratch reg. / new sb in inter-link-unit calls	Caller save
r13	sp	Lower end of current stack frame	-
r14	lr	link address / scratch register	Caller save
r15	pc	program counter	-

호출 규약이란:
컴퓨터 과학에서 어떻게 서브루틴이 그들의 caller로부터 변수를 받고, 어떻게 결과를 반환하는지에 대한 규약입니다.

플랫폼마다 이러한 규약을 다르게 사용하고 있고 컴파일러는 이를 보고 기계어를 만들어 냅니다.

부록 - APCS

글로벌 변수는 RW 데이터 영역에 할당

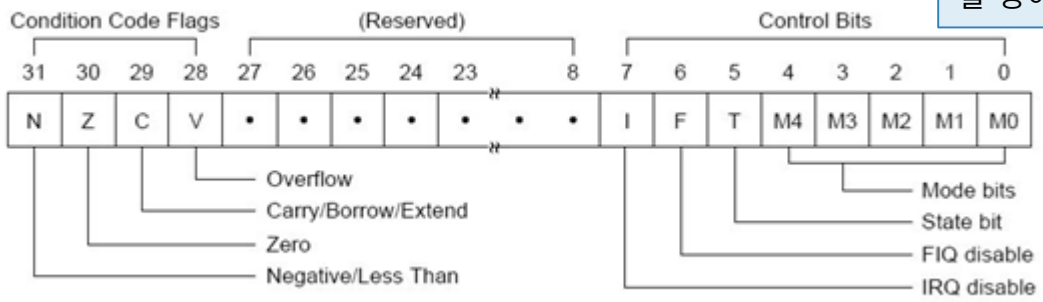
r0 ~ r3의 함수 인자 용 레지스터에 할당

4개 이상의 인자는 스택에 할당

```
Int gv1 = 100 ;

int function ( int a1, int a2, int a3, int a4, int a5, int a6, int a7)
{
        int v1, int v2, int v3, int v4, int v5, int v6, int v7 ;
        int v8, int v9, int v10 ;
        int ret;
        .
        .
        .
        .
        v10 = gv
        .
        .
        .
        ret = v7 – v1 ;
        return ret ;
}
```

r0를 통해서 결과 값 전달



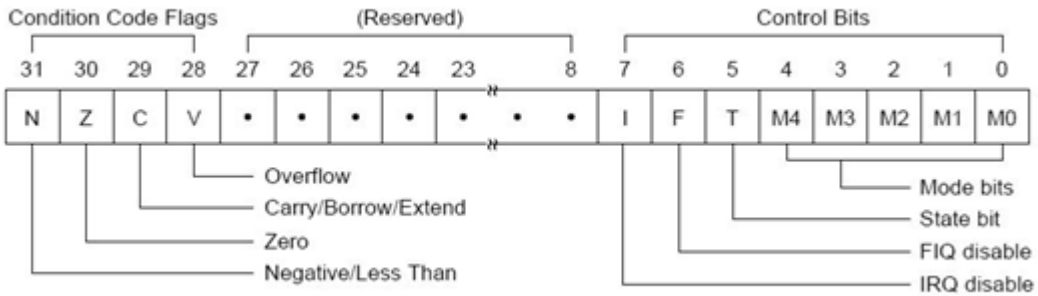
앞선 슬라이드에서 알 수 있듯이 r0 – r3 이 4개의 register는 함수 argument 값이 됩니다.
그 이상의 argument를 전달 하려면 스택을 이용합니다.

Local변수는 r4 – r10 혹은 r11까지도 사용합니다.
마찬가지로, 그 이상의 local변수를 사용하려면 스택을 이용합니다.

마지막으로, return값은 r0 register를 통해 전달합니다.

부록 - ARM7: A 32 bit CPSR

CPSR



ARM 코어는 내부 작동을 모니터링하고 제어하기 위해 CPSR을 사용합니다.

필 드	의 미
N	음수(Negative)플래그, 플래그 설정 연산 결과를 31 비트에 기록한다.
Z	제로 (Zero) 플래그, 플래그 설정 연산 결과가 0인 경우에 기록한다.
C	캐리 (Carry) 플래그, 덧셈에 대한 unsigned overflow를, 뺄셈에 대한 not-borrow를 기록한다. 쉬퍼트 회로에 의해서도 사용된다.
V	오버플로우(Overflow) 플래그, 플래그 설정 연산에 대해 signed overlfow를 기록한다.